Deep Learning Lab 2

Suraj Karki
Department of Engineering Science
University West
Trollhättan, Sweden
suraj.karki@student.hv.se

Nils Karlsson
Department of Engineering Science
University West
Trollhättan, Sweden
nils.Karlsson@student.hv.se

Nils Wikström
Department of Engineering Science
University West
Trollhättan, Sweden
nils.wikstrom@student.hv.se

***Index Terms*—Deep Learning, Laboration 2, PCA analysis, Dimensionality Reduction**

***Abstract*—The source code for this lab can be found at [1]. This lab focuses on PCA anlaysis and dimensionality reductions. PCA is used to reduce the dimensionality of the dataset and is then used to train a neural network. The results are compared with the original dataset. The upcoming lab focuses on building a pipeline to tune the hyperparameters of PCA and also the neural network. Finally, we use neural networks for classification and regression.**

I. Task 1: Principal Component Analysis (PCA)

A. 1- Load or create a dataset with more than 2 dimensions.

Data is loaded from sklearn datasets library. Iris dataset is used for this lab. It has 4 features (dimensions) and 3 classes.

B. 2- Find the first 2 principal components with and without using sklearn.

using sklearn, PCA from sklearn.decomposition is used to find the first 2 principal components. Without sklearn, PCA is implemented using numpy as shown below:

```python
# PCA without using sklearn
def pca_manual(X, n_components):
    # Center the data
    X_centered = X - np.mean(X, axis=0)

    # Compute covariance matrix
    covariance_matrix = np.cov(X_centered, rowvar=False) # sklearn uses rowvar=False by default

    # Eigen decomposition
    eigenvalues, eigenvectors = np.linalg.eigh(covariance_matrix) # sklearn uses svd decomposition but eigh is fine here for covariance
    matrix

    # Sort eigenvalues and eigenvectors
    sorted_indices = np.argsort(eigenvalues)[::-1]  # Descending order here, sklearn order by variance
    sorted_eigenvectors = eigenvectors[:, sorted_indices]

    # Select top n_components
    selected_eigenvectors = sorted_eigenvectors[:, :n_components]

    # Transform data
    X_reduced = np.dot(X_centered, selected_eigenvectors) # eqivalent to pca.fit_transform(X) in sklearn

    return X_reduced
```

Fig. 1: Manual PCA implementation using numpy.

Figure 1 shows the manual PCA implementation using numpy.

C. 3- Practice using PCA to preserve a certain percentage of variance.

Using sklearn PCA, we can preserve a certain percentage of variance by setting the n_components parameter to a float value between 0 and 1. For example, to preserve 95% of variance, we can set n_components=0.95.

D. 4- Train a classification or regression neural network by using: a. The Original dataset b. Principal components

Using `MLPClassifier` from `sklearn.neural _network`, we trained a neural network on both the original dataset and the principal components. Performance was evaluated using the accuracy score from `sklearn.metrics`. With the original data, an accuracy of 93% was achieved, while PCA-reduced data yielded 97%. The manual PCA implementation achieved an accuracy of 96.6%.

E. 5- Repeat part 4 using Kernel PCA (linear, sigmoid, RBF).

Using KernelPCA from sklearn.decomposition, we perofomed Kernel PCA with different kernels. For example, to use linear kernel, we can set kernel='linear'. The performance is compared using accuracy score from sklearn.metrics. With linear kernel 97% accuracy is achieved, sigmoid kernel yeilds only 36% accuracy and RBF kernel gives 96.6% accuracy.This shows that sigmoid activation does not perform well with this nearly linearly seperable dataset. It saturates easily and fails to learn complex patterns. It also overlaps the classes in the feature space. figure 2 shows the PCA results using sigmoid kernel.

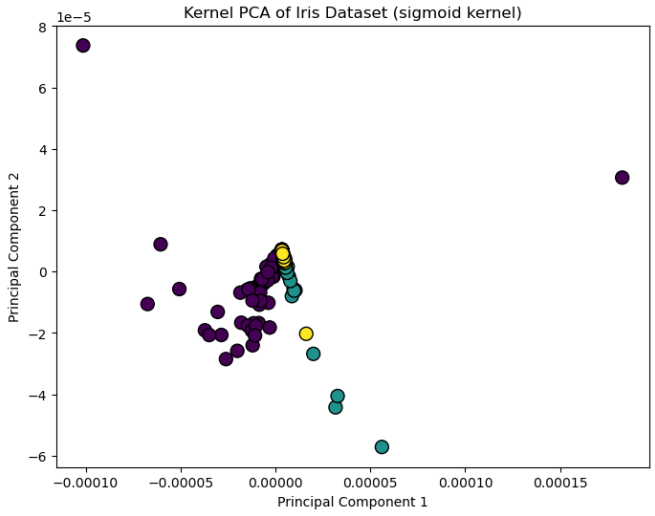

Fig. 2: Demonstrates the poor performance of sigmoid kernel in PCA.

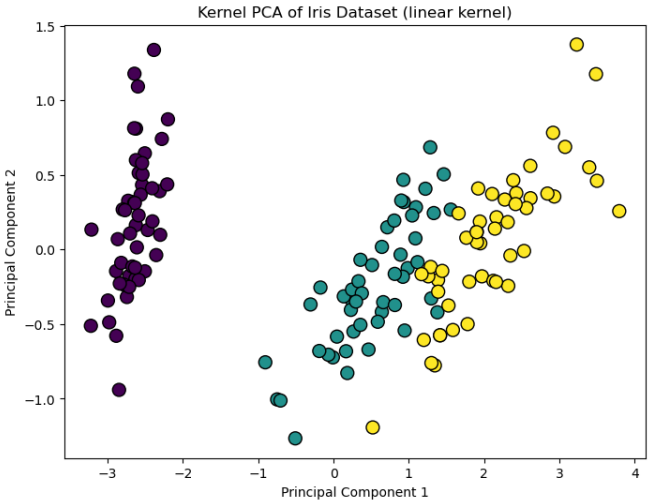

Fig. 3: PCA using linear kernel on Iris dataset (demonstration of kernel PCA).

figure 3 shows the PCA result using linear kernel on Iris dataset. Linear kernel and RBF kernel performs well on this dataset as it is nearly linearly seperable.

F. 6- Build a pipeline to tune the hyperparameters of Kernel PCA and also the neural network. Which hyperparameters can be tuned?

Using `Pipeline` and `GridSearchCV` from `sklearn.model_selection`, we built a pipeline to tune the hyperparameters of Kernel PCA and the MLPClassifier. The hyperparameters that can be tuned for kernel PCA are:

- number of components in Kernel PCA
- Kernel type (linear, sigmoid, RBF)
- gamma value for RBF and sigmoid kernels
- coef0 value for sigmoid kernel

For MLPClassifier, the hyperparameters that can be tuned are:

- Neural network solver (adam, sgd, lbfgs)
- Number of components
- Neural network hidden layer sizes
- Neural network activation function
- Neural network learning rate
- Neural network max iterations
- Neural network regualarization parameter (alpha)

II. Classification Problem

This IRIS model is a single-hidden-layer neural network with a tunable number of units (8–32) and activation function (relu or tanh), followed by a dropout layer and a softmax output layer for three-class classification. The model's hyperparameters, including dropout and learning rate, are optimised using Keras Tuner to achieve the best validation accuracy.

The result—training accuracy slightly lower than validation accuracy while validation reaches 1.0 and validation loss drops from 0.20 to 0.10, which is expected on a small dataset like IRIS, is shown in figure 4. Dropout reduces training performance by randomly deactivating neurons during training, whereas validation uses the full model without dropout.

In other hand MNIST dataset is more complex than iris, the model for the MNIST dataset consists of a **tunable convolutional layer** with 32–128 filters and 3×3 kernels followed by max pooling, a flatten layer, **1–4 fully connected dense layers** with tunable units (32–256) and activations (`relu` or `tanh`), and a dropout layer to reduce overfitting. The output layer has **10 neurons with softmax activation**, and the model is compiled with the Adam optimizer with a tunable learning rate. Figure 5 and 6 shows the output of MNIST model.

III. Regression Problem

After performing hyperparameter tuning on the neural network for the California housing dataset, we obtained an optimized configuration of the model. The first hidden layer contains 192 neurons (`units_1 = 192`), which is relatively large and allows the network to capture complex interactions among the input features. The network includes a second hidden layer (`second_layer = True`) with 32 neurons (`units_2 = 32`), which acts as a bottleneck to enhance generalisation while providing deeper feature representation. A small dropout rate of 0.1 (`dropout = 0.1`) is applied to reduce the risk of overfitting slightly. The learning rate is set to 0.01 (`learning_rate = 0.01`), which is fairly high and may accelerate learning, though it also induce oscillations see figure 7 and figure 8. The use of early stopping mitigates this risk by halting training once the validation performance plateaus. Overall, this combination of hyperparameters is reasonable for tabular regression on the California housing dataset, as it balances model complexity and generalisation effectively.

The model was retrained with a lower learning rate of 0.001. This adjustment yielded smoother curves of both mean absolute error and loss, as illustrated in Figures 10 and 9, indicating more stable convergence and reduced oscillations during training.

Overall, both models output similar results on the regression task.

References

[1] GitHubRepo, "Authors github repository for laborations," Online: https://github.com/Nilswik/DIL700/, accessed: 2026-02-04.

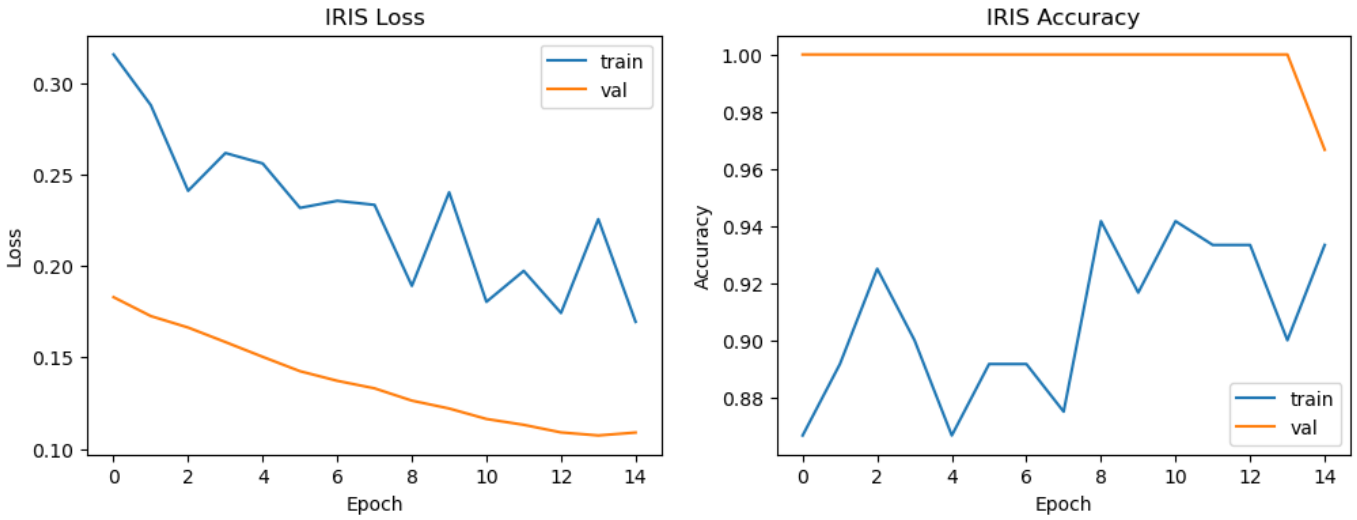

Fig. 4: Output of IRIS model

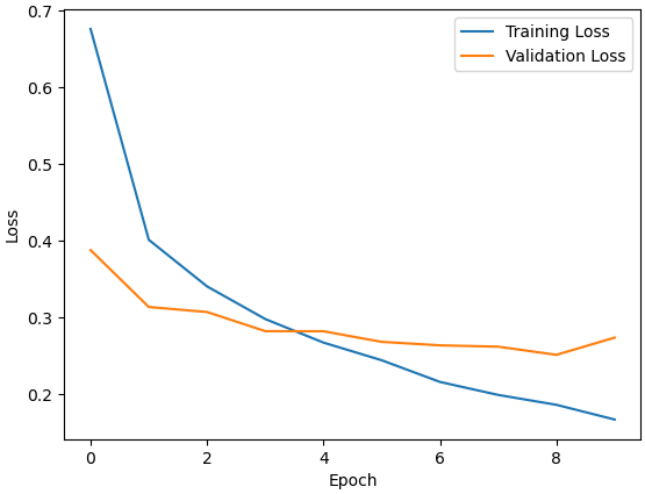

Fig. 5: Loss of MNIST model

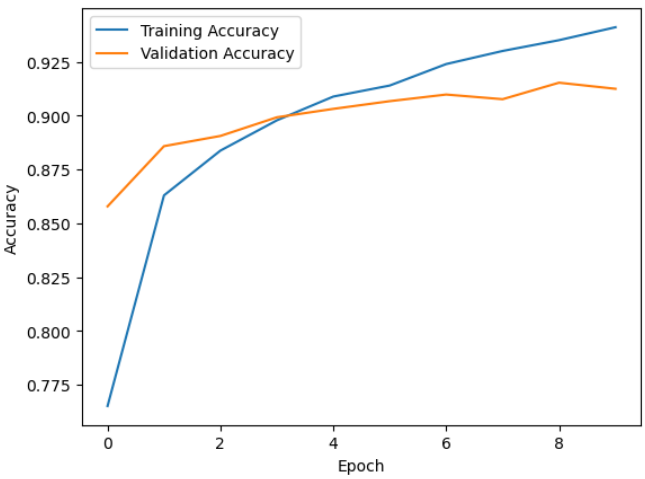

Fig. 6: Accuracy of MNIST model

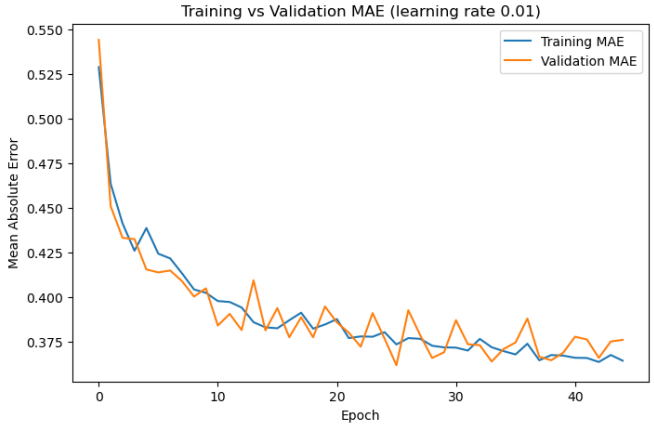

Fig. 7: overall training and test MAE and effect of learning rate on MAE with higher learning rate

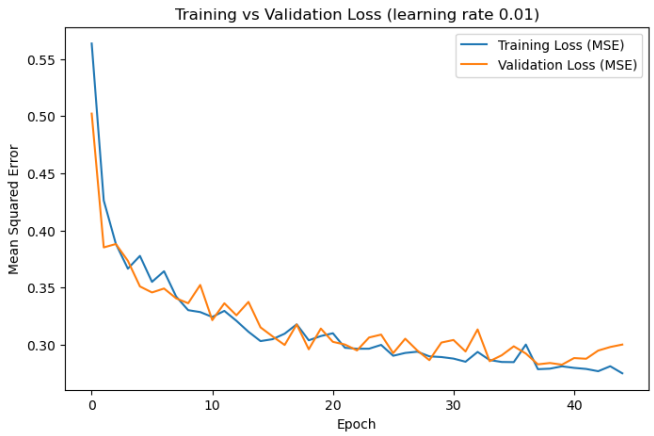

Fig. 8: Overall training and test loss and effect of learning rate on loss with higher learning rate

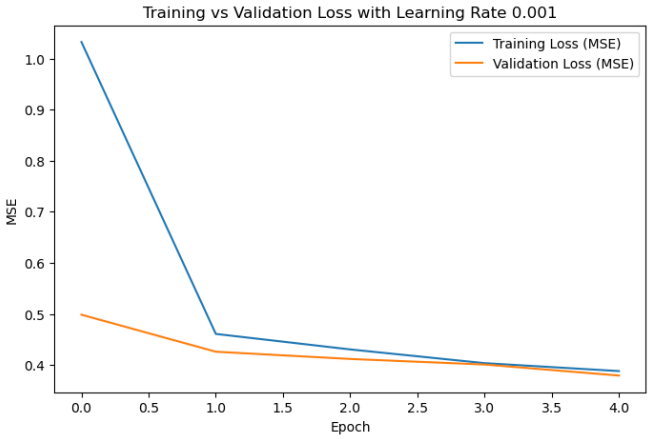

Fig. 9: Overall training and test loss and effect of learning rate on loss with lower learning rate

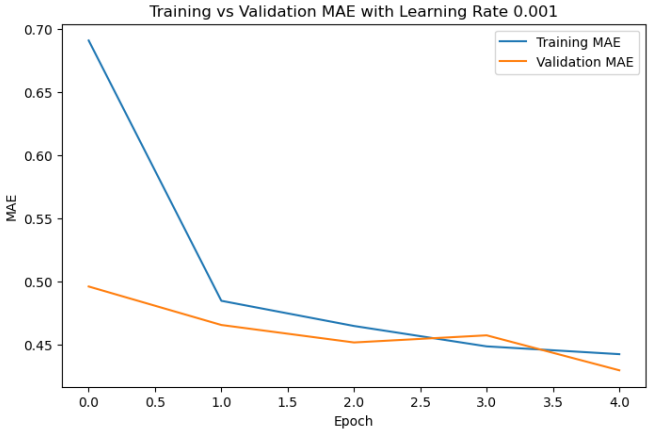

Fig. 10: Overall training and test MAE and effect of learning rate on MAEwith lower learning rate